

作业6-2

1. 什么是数据库的安全性？数据库系统一般提供哪5种安全性控制。请逐一详细描述或解释这5种安全性控制技术。

数据库的安全性是指保护数据库以防止不合法的使用所造成的数据泄露、更改或破坏。简单来说，就是确保只有“对的人”能做“对的事”，防止数据被窃取或篡改。

数据库系统通常提供以下5种安全性控制技术：

1. **用户标识与鉴别 (User Identification & Authentication)**：这是第一道大门。系统需要知道“你是谁”。通常通过用户ID和密码、生物特征（如指纹）或口令卡来验证用户身份。只有身份验证通过，才能进入系统。
2. **存取控制 (Access Control)**：这是核心机制。即使你进入了系统，也不代表你能看所有数据。系统会根据预定义的权限（如DAC自主存取控制或MAC强制存取控制），决定用户是否有权对某个表或字段进行读、写或修改操作。
3. **视图机制 (View Mechanism)**：这是一种隐蔽的保护手段。通过定义视图，只向用户展示他们需要看到的数据子集，而把敏感数据（如工资、密码等）隐藏起来，用户甚至不知道那些隐藏数据的存在。
4. **审计 (Auditing)**：这是一个“秋后算账”的监控机制。系统会自动记录用户对数据库的所有关键操作（谁、在什么时候、做了什么）到审计日志中。一旦发生安全事故，管理员可以通过查账来追究责任。
5. **数据加密 (Data Encryption)**：这是最后的防线。将原本明文的数据通过加密算法转换成密文存储。即使黑客窃取了数据库文件，没有解密密钥，看到也只是一堆乱码，无法理解数据内容。

2. 什么是事务？什么是事务的ACID特性？请逐一详细描述你对这4个特性的理解。

事务 (Transaction) 是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位。它是数据库恢复和并发控制的基本单位。

ACID特性是事务必须具备的四个特征：

1. **原子性 (Atomicity)**：强调“整体性”。事务就像原子一样不可分割。事务中的操作要么全部成功提交，要么如果中间出了错，就全部回滚（撤销），就像什么都没发生过一样，不允许只做了一半的情况存在。
2. **一致性 (Consistency)**：强调“守规矩”。事务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态。比如银行转账，无论怎么转，A账户少的钱必须等于B账户多的钱，总金额必须守恒，不能凭空消失或增加。

3. **隔离性 (Isolation)**: 强调“互不干扰”。当多个事务并发执行时, 一个事务的执行不应影响其他事务。在理想情况下, 每个事务感觉就像自己在独占系统一样, 看不到其他事务中间未提交的脏数据。
4. **持久性 (Durability)**: 强调“永久性”。一旦事务成功提交 (Commit), 它对数据库的改变就应该是永久的。哪怕下一秒系统崩溃、断电, 重启后数据依然要是修改后的样子, 不能丢失。

3. 数据库的并发操作可能出现哪些不一致问题? 怎样才能有针对性的解决这些不一致问题?

如果不加控制地让多个事务同时操作数据, 通常会出现以下三类数据不一致问题:

1. **丢失修改 (Lost Update)**: 两个事务同时读入同一数据并进行修改, 后提交的事务覆盖了先提交的事务的修改, 导致前一个事务的操作像没发生过一样。
2. **不可重复读 (Non-repeatable Read)**: 事务A读取数据后, 事务B修改或删除了该数据, 事务A再次读取时发现数据变了或没了, 导致前后读取不一致。
3. **读“脏”数据 (Dirty Read)**: 事务A读取了事务B修改但尚未提交的数据, 后来事务B因为某种原因撤销了, 那么A读到的就是不存在的错误数据。

解决办法:

核心手段是封锁 (Locking) 技术。

- 系统引入了**排他锁 (X锁/写锁)** 和 **共享锁 (S锁/读锁)**。
- 通过遵守**三级封锁协议**来针对性解决:
 - 一级封锁协议 (修改前加X锁, 直到事务结束) 解决“丢失修改”。
 - 二级封锁协议 (在一级基础上, 读取前加S锁, 读完立刻释放) 解决“读脏数据”。
 - 三级封锁协议 (在一级基础上, 读取前加S锁, 直到事务结束才释放) 解决“不可重复读”。
- 也就是通过强制排队或限制读写权限, 打破并发操作的交叉干扰。

4. 数据库日志的作用是什么? 每一条日志记录了什么? 登记日志需要遵循哪两条原则, 为什么?

日志的作用: 主要用于数据库的**恢复 (Recovery)**。它是数据库在出现故障时, 将数据还原到正确状态的依据。此外, 日志也用于审计和分析系统性能。

日志记录的内容: 一般包括: 事务标识 (是哪个事务)、操作类型 (插入、删除或修改)、操作对象 (哪个记录)、**旧值 (Before Image)** (用于撤销/UNDO) 和 **新值 (After Image)** (用于重做/REDO)。

登记日志必须遵循的两条原则（WAL协议 - Write-Ahead Logging）：

计算机

1. 登记的次序严格按并发事务执行的时间次序。
2. 必须先写日志，后写数据库。（Write-Ahead）

为什么（特别是第二条）？

这是为了确保故障恢复的可能性。

- 如果先写了数据库，但在写日志之前系统崩溃了，那么数据库里已经有了修改，但日志里没有记录。重启后系统不知道该操作发生过，无法进行撤销（UNDO），破坏了原子性。
- 反之，如果先写了日志，即使写数据库前崩溃了，重启后系统可以通过日志知道“有个操作没完成”，从而执行撤销，保证数据一致性。只有日志先落盘，才能保证在任何灾难下都能恢复。

5. 数据库运行过程中可能产生哪3类故障？针对不同的故障，应该如何进行恢复？

数据库运行中主要有以下3类故障：

1. 事务内部的故障：

- 现象：运算溢出、违反约束、死锁导致事务被杀掉。
- 恢复：利用日志文件进行**撤销 (UNDO)**。系统反向扫描日志，找到该事务的更新操作，将数据恢复到修改前的“旧值”。

2. 系统故障（软故障）：

- 现象：CPU故障、操作系统崩溃、突然停电。内存中的数据丢失，但硬盘数据还在。
- 恢复：重启时执行**UNDO + REDO**。
 - 对于崩溃时未提交的事务，执行**UNDO**（撤销），保证原子性。
 - 对于崩溃前已提交但在内存中还没来得及写回磁盘的事务，执行**REDO**（重做），保证持久性。

3. 介质故障（硬故障）：

- 现象：磁盘损坏、磁头碰撞、火灾水淹。物理存储被破坏，数据全丢。
- 恢复：**重装数据库/副本 + 重做日志**。
 - 第一步：装入最新的数据库**后备副本 (Backup)**，将数据库恢复到备份时的状态。
 - 第二步：装入备份后的**日志文件副本**，执行**REDO**（重做），将自备份以来所有已提交的事务重新执行一遍，把数据追回至故障发生前的一瞬间。